

Chapitre 9 Fonctions

Les fonctions regroupent les instructions qui ont un lien logique. Elles sont très utilisées car elles organisent et structurent le code des programmes. Elles rendent ainsi les programmes plus faciles à relire et donc à corriger, et plus fiables car elles limitent les erreurs.

En C et en C++, la plupart des instructions des programmes sont organisées en fonctions. Le programme principal est lui-même une fonction, dont l'identificateur est `int main()`, spécifiant que cette fonction particulière (principale) doit être exécutée en premier.

Les fonctions sont faciles à utiliser. La seule difficulté, que l'on retrouve dans tous les langages de programmation, réside dans la maîtrise des différentes méthodes de déclaration des paramètres (valeurs passées entre parenthèses pour utiliser les fonctions). Contrairement à d'autres langages, le C++ simplifie grandement le passage des paramètres pour l'utilisateur des fonctions.

Déclaration et définition

Pour utiliser ou définir une fonction, il est préférable de l'avoir déclarée au préalable. Ainsi, on donne la possibilité au compilateur de lever d'éventuelles ambiguïtés et de réaliser un maximum de vérifications, notamment sur le nombre, le type ou la valeur de retour de la fonction.

Déclarer une fonction revient à décrire le prototype de celle-ci, c'est à dire énumérer le type des paramètres qu'elle utilise et la valeur qu'elle retourne. Ainsi, lorsque cela est possible, le compilateur est en mesure de prévoir les conversions de types nécessaires à une transmission correcte des paramètres et de la valeur renvoyée.

Déclaration du prototype d'une fonction

La syntaxe de **déclaration** d'une fonction est la suivante. Le nom de la fonction (`identificateur`) doit être précédé du `type` de la valeur retournée par la fonction, et suivi (entre parenthèses) de la liste de ses paramètres.

Syntaxe

```
type identificateur (type1, type2, type3, ...);
```

ou

```
type identificateur (type1 param1, type2 param2, type3 param3, ...);
```

`type` est le type de la valeur de retour de la fonction, `type 1`, `type 2`, `type 3`, sont les types des paramètres de la fonction, et `param1`, `param2`, `param3` les sont identificateurs des paramètres de la fonction.

Exemple de déclaration de prototypes de fonctions

```
double sinus (double);
int premier (int);
void pgcd (int, int);
double moyenne (int, double*);
int init(void);
```

Remarques

- Si une fonction ne retourne pas de valeur, elle doit être précédée du mot réservé `void`.
- Si une fonction n'a pas de paramètre, il suffit de la déclarer avec une liste vide `()`, ou avec le type `void` entre parenthèses `(void)` pour préciser que les paramètres n'ont pas été oubliés.
- S'il est possible de préciser les identificateurs des paramètres lors de la déclaration des fonctions, il est courant de ne pas le faire afin d'éviter toute différence avec les identificateurs des paramètres utilisés lors de la définition des fonctions.



- Avec certains compilateurs, l'utilisation d'une fonction non déclarée au préalable conduit à la définition d'un prototype par défaut, induit par le type des paramètres rencontrés lors du premier appel.
- De même, si le type de la valeur retournée par la fonction n'est pas défini, le type entier `int` est présumé. Pour lever toute ambiguïté, il est donc préférable de déclarer les fonctions avant de les utiliser.

Définition d'une fonction

La syntaxe pour **définir** une fonction est la suivante. L'entête de la fonction précise le type de retour, l'identificateur de la fonction, et la liste entre parenthèses des types et identificateurs de ses paramètres.

Vient ensuite le corps de la fonction constitué des instructions de la fonction placées entre accolades `{ }`. Lorsque la fonction retourne une valeur, il faut utiliser, en n'importe quel point de cette fonction mais de préférence à sa fin pour plus de clarté, l'instruction `return` qui permet de sortir et retourner une valeur de la fonction.

Syntaxe

```
type identificateur (type1 param1, type2 param2, type3 param3, ...)
{
    instructions de la fonction;
    ...
    return valeur;           // valeur est de type "type"
}
```

Remarque

- Attention de ne pas insérer de `;` à la fin de l'entête de la fonction (cela serait considéré comme une déclaration de la fonction).

Exemple

La fonction `divisible` retourne 1 si un nombre entier `nNum` est divisible par un second nombre entier `nDen`, c'est-à-dire si le reste de la division de `nNum` par `nDen` est égal à zéro.

```
int divisible(int, int);           // déclaration (ou prototype) de la fonction

int divisible (int nNum, int nDen) // définition de la fonction
{
    if ( (nNum%nDen) == 0 ) return 1;
    else return 0;
}
```

Type prédéfini référence

Le type prédéfini référence est une particularité du C++, qui n'existe pas en C. Le type référence permet de déclarer une variable qui référence une autre variable. Cela signifie que la variable référence peut être utilisée en tout point à la place de la variable référencée.

Pour déclarer une variable de type référence, on fait suivre le type de la variable référence par le caractère &, et on initialise immédiatement la variable référence par une variable référencée.

Syntaxe

```
type   tVariable = (type) 0;           // déclaration classique d'une variable
type& rVariable = tVariable;          // déclaration d'une variable référence
                                         // rVariable sur la variable tVariable
                                         // tVariable est la variable référencée
```

Remarques

- Une variable référence doit être initialisée immédiatement lors de sa déclaration (en utilisant l'opérateur d'affectation = suivi par l'identificateur de la variable référencée). Cette affectation est définitive et ne peut évoluer pendant l'exécution du programme.
- Une variable référence d'une autre variable peut être utilisée indifféremment à la place de la variable référencée.
- La création d'une variable référence ne s'accompagne pas de réservation supplémentaire d'emplacement mémoire. L'opérateur `sizeof()` utilisé avec une variable référence renvoie la taille mémoire occupée par la variable référencée.
- On peut voir la déclaration d'une variable référence comme la déclaration d'un nouvel identificateur (nouveau nom) pour la variable référencée. Le sens (ont dit la sémantique) des variables référence est la même que les références de la vie quotidienne.
- L'usage des variables référence est quasi exclusivement consacré à la transmission des paramètres de sortie des fonctions.
- Pour améliorer la lisibilité et la compréhension du code, il est courant d'accoler le caractère & directement au type (`type& rVariable = tVariable;`) plutôt que de l'accoler à l'identificateur de la variable (`type &rVariable = tVariable;`) afin de limiter la confusion avec l'opérateur `&tVariable` qui retourne l'adresse de la variable `tVariable`.



Attention, variable référence et variable référencée correspondent au même emplacement mémoire. Toute modification du contenu de l'emplacement mémoire par la variable référence (respectivement par la variable référencée) impacte le contenu de la variable référencée (respectivement de la variable référence) puisqu'il s'agit de la même zone mémoire !

Exemple de déclaration et d'utilisation de variables références

```
int nI = 3;                // variable entière initialisée avec la valeur 3
int& rJ = nI;              // déclaration de rJ, variable référence sur nI
cout << rJ << endl;       // affiche 3 à l'écran (contenu de nI)

rJ = 2;                   // modifie le contenu de rJ (et donc celui de nI)
cout << nI << endl;       // affiche 2 à l'écran

int nK = 8;
rJ = nK;                  // le contenu de nI prend la valeur de nK (8)
cout << sizeof (rJ);       // affiche la taille de la mémoire occupée par nI
```

Transmission des paramètres des fonctions

Si écrire des fonctions en C++ n'est pas difficile, choisir le modèle de transmission des paramètres à mettre en œuvre (parmi trois possibilités) peut le paraître au premier abord. C'est pourtant nécessaire, en C++ comme pour tout langage de programmation. Pour faciliter ce choix, il est nécessaire de distinguer si chaque paramètre de la fonction est un paramètre :

- d'entrée
- d'entrée multivalué,
- parmi trois possibilités
- d'entrée/sortie

Il s'agit autant d'apprendre et maîtriser les syntaxes C++ propres à l'écriture de chaque mode de transmission, mais également d'apprendre à identifier le rôle de chaque paramètre afin de retenir le mode de transmission des paramètres adapté.

Un paramètre d'entrée d'une fonction est un paramètre dont le contenu ne sera pas modifié par la fonction. Il s'agit d'un élément utile pour l'exécution de la fonction, fourni par le programme d'appel de la fonction, mais sans incidence pour ce programme d'appel. Exemple : `double sin (double);` le paramètre de la fonction (l'angle dont on veut connaître le sinus), n'est pas modifié par la fonction `sin()`.

On distingue les paramètres d'entrée simples, constitués d'une seule valeur, généralement une variable d'un type prédéfini, et les paramètres d'entrée multivalués, composés de plusieurs valeurs, généralement des objets de classes. Exemple : `double Module (const CComplex&);` le paramètre de la fonction (le complexe dont on veut calculer le module), représente deux valeurs : la partie réelle et la partie imaginaire du complexe.

Inversement, un paramètre de sortie d'une fonction est un paramètre dont la valeur sera modifiée dans la fonction. Il s'agit d'une variable fournie par le programme appelant dont la fonction transformera la valeur pour qu'elle soit utilisée ensuite par le programme appelant. Exemple : `void Aleatoire(double)` ; qui initialise par une valeur aléatoire le réel passé en paramètre.

Les paramètres d'entrée / sortie d'une fonction sont des paramètres à la fois d'entrée et de sortie. Ce sont des paramètres fournis à la fonction, modifiés par la fonction, et retournés modifiés au programme appelant. C'est par exemple le cas des fonctions dont l'objet est de modifier le contenu d'une variable. Exemple : `void ValeurAbsolue(double&);` qui remplace le réel passé en paramètre par sa valeur absolue.

Une fois le type de chaque paramètre identifié, il est facile de mettre en œuvre le mode de transmission adapté à chaque cas, selon le tableau suivant.

Type de paramètre	Mode de transmission du paramètre
Paramètre d'entrée	Passage (ou transmission) par valeur
Paramètre d'entrée multivalué	Passage par référence constante
Paramètre de sortie	Passage (ou transmission) par référence
Paramètre de sortie	

Tableau 15 : choix du mode de transmission en fonction du type des paramètres des fonctions.

Paramètres d'entrée : transmission par valeur

Par défaut, le passage des paramètres dans les fonctions se fait par **valeur**. Ceci signifie que ce sont les valeurs des paramètres qui sont transmises aux fonctions, et qu'aucune modification de ces valeurs dans

les fonctions n'est répercutée dans la fonction d'appel (lors de l'appel d'une fonction, les valeurs paramètres sont recopiés dans une nouvelle zone mémoire libérée à la sortie de la fonction). Cette transmission des paramètres par valeur est utilisée pour les paramètres d'entrée, qui ne sont pas modifiés dans les fonctions.

Syntaxe

```
type identificateur (type1 param1, type2 param2, ...)
```

Remarques

- Un paramètre transmis par valeur dans la fonction est déclaré comme une variable classique. Un espace de la taille du type du paramètre est réservé dans la mémoire. On y accède simplement grâce à l'identificateur du paramètre.
- Transmettre les paramètres d'entrée par valeur garantit que la fonction ne peut modifier les valeurs des paramètres du programme appelant. C'est une sécurité qu'il faut utiliser pour limiter les effets de bord, c'est à dire des erreurs intempestives qui se propagent tout au long d'un programme.
- La transmission par valeur des paramètres des fonctions implique une consommation supplémentaire de mémoire et de temps processeur de l'ordinateur. En effet, tout paramètre transmis par valeur a un emplacement nouveau réservé dans la mémoire, et s'accompagne d'une copie de la valeur du paramètre initial dans le nouvel emplacement mémoire.
- L'espace mémoire réservé pour le stockage de chaque paramètre transmis par valeur est libéré à la ligne de l'accolade de fin de la fonction, dans l'ordre inverse de la déclaration.
- La valeur de retour d'une fonction peut aussi correspondre à une valeur. Cela autorise à mettre une fonction dans le membre de droite d'une affectation (à la place d'une valeur), mais pas dans le membre de gauche d'une affectation (qui doit être une variable).

Exemple de paramètres transmis par valeur

```
int Somme (int, int);           // déclaration de la fonction Somme()

int Somme (int nA, int nB)      // définition de la fonction Somme()
{
    int nT = nA + nB;           // Étape 2
    nB *= 2;                    // Étape 3
    return nT;
}

int main (void)                 // fonction principale
{
    int nI = 3;
    int nJ = 4;
    int nS = 0;                 // Étape 1

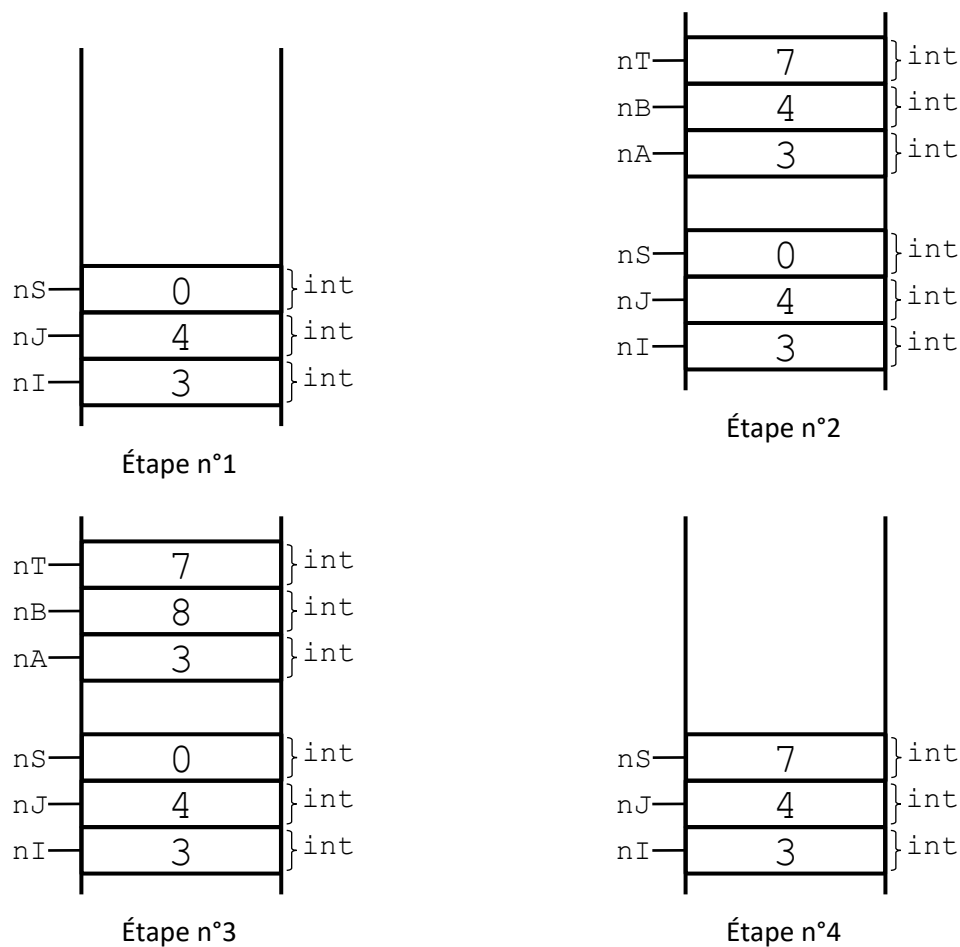
    nS = Somme(nI,nJ);          // Étape 4
}
```

- A l'Étape 1, les 3 variables entières `nI`, `nJ` et `nS` de la fonction `int main(void)` sont réservées en mémoire, et initialisées respectivement par les valeurs 3, 4 et 0.
- A l'Étape 2, les deux paramètres `nA` et `nB` de la fonction `int Somme(int, int);` sont réservés en mémoire, et initialisés respectivement avec les valeurs des deux variables `nI` (3) et `nJ` (4). La

variable locale `nT` est réservée en mémoire et initialisée avec la somme des valeurs des paramètres `nA` et `nB` (3+4).

- A l'Étape 3, la valeur du paramètre `nB` est multipliée par 2 (8).
- A l'Étape 4, les deux paramètres `nA` et `nB` de la fonction `int Somme(int, int);` sont libérés de la mémoire, ainsi que variable locale `nT`, car la portée des variables est limitée au bloc d'instructions `{}` dans lequel elles ont été déclarées (ici le corps de la fonction `int Somme(int, int);`). Il ne reste plus en mémoire que les variables initiales de la fonction `int main(void)` (`nI`, `nJ` et `nS`). `nS` contient maintenant la valeur de retour de la fonction `int Somme(int, int);` (7).

Évolution de l'occupation mémoire des variables du programme



Paramètres de sortie ou d'entrée / sortie : transmission par référence

Le passage des paramètres de sortie ou d'entrée / sortie des fonctions se fait par **référence**. Dans la fonction, les paramètres sont des références sur les variables du programme appelant passées en paramètres. On accède donc directement dans la fonction aux variables du programmes appelant, dont les valeurs peuvent ainsi être modifiées dans la fonction.

Syntaxe

```
type identificateur (type1& param1, type2& param2, ...)
```

Remarque

- La transmission par référence des paramètres des fonctions est très utilisée, car le contenu des paramètres est très souvent modifié dans les fonctions. En C++, la transmission par référence est simple à écrire dans les fonctions et simple à utiliser dans les programmes d'appel des fonctions.
- La transmission par référence des paramètres économise la mémoire et le processeur de l'ordinateur, puisque les paramètres référencent directement des variables existantes. Contrairement à la transmission des paramètres par valeur, il n'y a pas de réservation supplémentaire d'emplacement mémoire ni de copie de valeur dans cet emplacement mémoire.
- La valeur de retour d'une fonction peut être aussi de type référence, ce qui autorise à mettre une fonction dans le membre de gauche d'une affectation (à la place d'une variable), ou bien dans le membre de droite (c'est alors la valeur contenue dans la variable qui est utilisée).

Exemple de paramètres transmis par référence

```

void Echange (int&, int&);           // déclaration de la fonction Echange

void Echange (int& n1, int& n2)     // définition de la fonction Echange
{
    int nT = n1;                    // Étape 2
    n1 = n2; n2 = nT;               // Étape 3
}

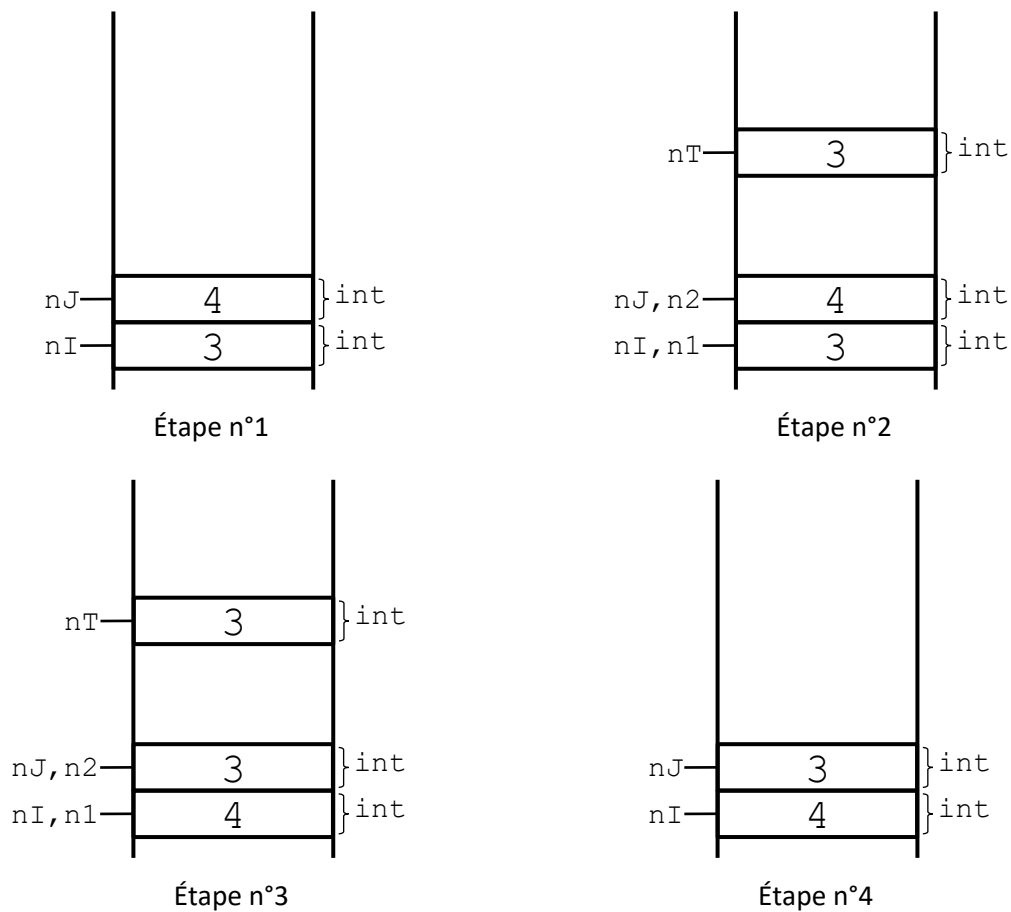
int main (void)                    // fonction principale
{
    int nI = 3;
    int nJ = 4;                    // Étape 1

    Echange(nI, nJ);                // Étape 4
}

```

- A l'Étape 1, les 2 variables entières `nI` et `nJ` de la fonction `int main(void)` sont réservées en mémoire et initialisées respectivement par les valeurs 3 et 4.
- A l'Étape 2, les deux paramètres référence `n1` et `n2` de la fonction `void Echange(int&, int&);` référencent respectivement les 2 paramètres `nI` et `nJ` du programme appelant, sans réservation de mémoire supplémentaire (variables références). La variable locale `nT` est réservée en mémoire et initialisée avec la valeur du paramètre `n1` (3) (donc la valeur de la variable `nI` puisque `n1` référence `nI`).
- A l'Étape 3, le contenu du paramètre `n1` (donc de la variable `nI` puisque `n1` référence `nI`) est remplacé par la valeur (4) du paramètre `n2` (donc de la variable `nJ` puisque `n2` référence `nJ`). Le contenu du paramètre `n2` (donc de la variable `nJ` puisque `n2` référence `nJ`) est remplacé par la valeur de la variable locale `nT` (3).
- A l'Étape 4, les deux paramètres références `n1` et `n2` de la fonction `void Echange(int&, int&);` sont supprimés, et la variable locale `nT` est libérée, car la portée des variables est limitée au bloc d'instructions `{}` dans lequel elles ont été déclarées (ici le corps de la fonction `void Echange(int&, int&);`). Il ne reste plus en mémoire que les variables initiales de la fonction `int main(void)` (`nI` et `nJ`), dont les contenus ont été échangés grâce au code de la fonction `void Echange(int&, int&);`.

Évolution de l'occupation mémoire des variables du programme



Paramètres d'entrée multivalués : transmission par référence constante

Les paramètres d'entrée n'étant pas modifiés par les fonctions, ils sont transmis par valeur. Cela consomme de la mémoire (pour réserver les paramètres) et du temps processeur (pour recopier les valeurs dans les paramètres). Cela a très peu d'importance pour des paramètres simples de type prédéfini. Par contre, cette réservation supplémentaire de mémoire peut devenir un inconvénient lorsque les paramètres d'entrée occupent beaucoup de place en mémoire. C'est par exemple le cas lorsque ces paramètres sont constitués de plusieurs valeurs (on parle alors de paramètres multivalués).

Pour transmettre des paramètres d'entrée qui occupent beaucoup de mémoire, sans recopie temporaire (optimisation de la mémoire et de l'exécution du programme), on utilise la transmission par référence (qui économise la mémoire et les temps de recopie), en spécifiant que les paramètres doivent rester constants dans la fonction (et donc ne pas être modifiés). Pour cela, on utilise le mot réservé `const` avant la déclaration de chaque paramètre. Ainsi, les paramètres, bien que transmis par référence, ne peuvent être modifiés dans la fonction.

Syntaxe

```
type identificateur (const type1& param1, const type2& param2, ...)
```

Exemple de passage de paramètre d'entrée multivalué (classe `CComplex`) par référence constante

```
void CComplex::Amplitude (const CComplex& cplx);
```

Valeurs par défaut des paramètres des fonctions

En C++, il est possible d'omettre certains paramètres transmis par valeur lors de l'appel d'une fonction. Il suffit pour cela d'utiliser l'opérateur d'affectation au moment de la déclaration des fonctions pour fixer des valeurs choisies par le programmeur. Lorsque les valeurs sont omises, ce sont ces valeurs qui sont utilisées pour initialiser le contenu des paramètres.

Syntaxe

```
type identificateur (type1 param1, type2 param2 = val2, type3 param3 = val3, ...);
```

Remarques

- Les paramètres initialisés par défaut doivent obligatoirement être les derniers de la liste de paramètres. De même, lors de l'appel de la fonction, on ne peut omettre que le ou les derniers paramètres, parmi ceux qui ont une valeur par défaut.
- Il est possible, dans la déclaration d'une fonction, de définir des paramètres qui n'étaient pas déjà définis.
- Il n'est pas possible d'utiliser l'initialisation par défaut pour les paramètres transmis par référence.

Exemple

```
double Division (double = 0., double = 1.);    // déclaration de la fonction

double Division (double dA, double dB)        // définition de la fonction
{
    if (b != 0) return (dA / dB);
    else return dA;
}

int main (void)                                // utilisation de la fonction
{
    double dC = Division ();                   // dC = 0.0 / 1.0 = 0.0
    double dD = Division (3.0);               // dD = 3.0 / 1.0 = 3.0
    double dE = Division (5.0, 2.0);         // dE = 5.0 / 2.0 = 2.5
}
```

Paramètres de la ligne de commande

Dans un programme C++, il est possible de récupérer les paramètres de la ligne de commande, c'est à dire des valeurs passées en lançant le programme suivi de ces valeurs en ligne de commande dans une fenêtre de type terminal.

Pour cela, il suffit de déclarer deux paramètres à la fonction `int main (int argc, char* argv[]);`.

Le premier paramètre, de type entier (`int`), `argc`, contient le nombre de champs qui composent la ligne de commande (au minimum 1, le nom du programme). Le second paramètre, de type tableau de chaîne de caractères (`char* []`), `argv`, contient ces éléments sous la forme d'un tableau de `argc` chaînes de caractères. `argv[0]` contient le nom du programme. `argv[i]` contient la *i*^{ème} valeur passée à la suite du nom du programme, pour *i* allant de 1 à `argc-1`.

Le séparateur des paramètres est une suite non vide de caractères d'espacement : espaces ou tabulations.

Exemple

L'exemple suivant affiche tous les champs de la ligne de commande.

```
#include <iostream.h>
void main (int argc, char* argv[])
{
    cout << "argc = " << argc;           // nombre de paramètres
    for (int i = 0 ; i < argc ; ++i)      // affichage des paramètres
        cout << "argv[" << i << "] = " << argv[i] << endl;
}
```

En lançant le programme en ligne de commande `.\appli.exe a b c`, on obtient l'affichage :

```
C:\users\chris> .\appli.exe a b c

argc = 4
argv[0] = .\appli.exe
argv[1] = a
argv[2] = b
argv[3] = c
```

Transmission par adresse des paramètres de sortie ou d'entrée/sortie (langage C)

En C, le type référence n'existe pas. Il n'est donc pas possible de transmettre les paramètres de sortie ou d'entrée / sortie des fonctions par référence. On utilise à la place une transmission par valeur des adresses de ces paramètres de sortie ou d'entrée / sortie. Ainsi, on accède directement dans les fonctions aux paramètres passés aux fonctions par le programme qui les appellent.

En transmettant l'adresse des paramètres dans le programme appelant, il est possible, grâce à l'opérateur de déréréférenciation `*`, d'accéder dans les fonctions à l'emplacement mémoire des paramètres. On ne transmet plus les valeurs des paramètres, mais leurs adresses. A partir de ces adresses, on accède aux contenus grâce à l'opérateur `*`. Toute modification de la valeur est donc automatiquement répercutée dans la fonction d'appel, puisque c'est l'emplacement mémoire dont l'adresse est passée aux fonctions qui est modifié. Ce mécanisme de transmission par adresse est utilisé pour les paramètres de sortie et d'entrée / sortie des fonctions.

Syntaxe

```
type identificateur (type1*, type2*, ...);    // déclaration de la fonction

type identificateur (type1* param1, type2* param2, ...) // définition
{                                                    // de la fonction
    ...
}

int main (void)
{
    type1 p1;
    type2 p2;
    identificateur (&p1, &p2);
}
```

Remarques

- Dans tout le corps de la fonction, `param'i'` représente l'adresse du paramètre, et `*param'i'` l'emplacement mémoire (contenu) pointé par `param'i'`.

- Le passage des paramètres par adresse du C est une transmission par valeur des adresses des paramètres.
- Contrairement à la transmission des paramètres par référence, qui est d'un usage très simple en C++, il faut utiliser l'opérateur `&` pour passer l'adresse des paramètres de sortie ou d'entrée / sortie. Lors de l'appel de la fonction, il faut prendre garde à transmettre les adresses des paramètres en les faisant précéder de l'opérateur `&` (adresse du paramètre).
- L'utilisation de la transmission par adresse des paramètres de sortie ou d'entrée / sortie est vivement déconseillée en C++, afin de réduire les risques d'erreur (`&` oubliés lors de l'appel de la fonction ou `*` oubliés lors de l'utilisation des paramètres dans la fonction).

Exemple de paramètres transmis par adresse

```
void Echange (int*, int*);           // déclaration de la fonction Echange

void Echange (int* p1, int* p2)     // définition de la fonction Echange
{
    int nT = *p1;                   // Étape 2
    *p1 = *p2; *p2 = nT;           // Étape 3
}

int main (void)                     // fonction principale
{
    int nI = 3;
    int nJ = 4;                     // Étape 1

    Echange (&nI, &nJ);             // Étape 4
}
```

- A l'Étape 1, les 2 variables entières `nI` et `nJ` de la fonction `int main(void)` sont réservées en mémoire, et initialisées respectivement par les valeurs 3 et 4.
- A l'Étape 2, les deux paramètres pointeurs `pA` et `pB` de la fonction `void Echange(int*, int*);` sont réservés en mémoire, et initialisés respectivement avec les adresses des deux variables `nI` et `nJ` du programme appelant. La variable locale `nT` est réservée en mémoire et initialisée avec le contenu de la variable (`*p1`) dont l'adresse est dans `p1`, soit le contenu de la variable `nI` (3).
- A l'Étape 3, le contenu de la variable (`*p1`) dont l'adresse est dans `p1` (soit le contenu de la variable `nI` puisque `p1` contient l'adresse de `nI`) est remplacé par le contenu de la variable (`*p2`) dont l'adresse est dans `p2` (soit le contenu de la variable `nJ` puisque `p2` contient l'adresse de `nJ`) (4). Le contenu de la variable (`*p2`) dont l'adresse est dans `p2` (soit le contenu de la variable `nJ` puisque `p2` contient l'adresse de `nJ`) est remplacé par la valeur de la variable locale `nT` (3).
- A l'Étape 4, les deux paramètres pointeurs `p1` et `p2` de la fonction `void Echange(int*, int*);` sont supprimés, et la variable locale `nT` est libérée, car la portée des variables est limitée au bloc d'instructions `{}` dans lequel elles ont été déclarées (ici le corps de la fonction `void Echange(int*, int*);`) Il ne reste plus en mémoire que les variables initiales de la fonction `int main(void)` (`nI` et `nJ`), dont les contenus ont été échangés grâce au code de la fonction `void Echange(int*, int*);`.

Évolution de l'occupation mémoire des variables du programme

